

论批处理系统的设计与实现

⚠ **关于本文中的数字** 本文的量化数据（作业数量、执行时长、失败率等）均为**示例值**，用于展示论文应有的完整度与写法。**架构类数字**（表数量、接口数、配置值）已从项目源码核实，可直接使用；**业务规模类数字**（门店数、用户数、业务量、改善幅度）在项目资料中尚未确定，属示例值，背诵前请替换为你的真实数据。文末附有「待替换数字清单」，便于逐项核对。

试题

论批处理系统的设计与实现

批处理系统是指按照预定规则，在无需人工干预的情况下，对大量数据执行周期性或触发式处理的系统。批处理系统广泛应用于数据同步、报表生成、账单结算、消息批量推送等场景。批处理系统的设计需要重点考虑作业划分与调度、大数据量的处理效率、失败重试与断点续跑、与在线业务的资源隔离以及执行过程的可观测性等问题。

请围绕“批处理系统的设计与实现”论题，依次从以下三个方面进行论述：

- 简要叙述你参与设计和开发的批处理系统，以及你所担任的工作；
- 详细论述你在该系统中的批处理设计，包括作业划分、调度策略、容错与重跑机制；
- 分析你在批处理系统设计与实施中遇到的问题及解决措施。

摘要

本文以我参与建设的 H.I.S. BASE HOME 旅游业务平台中的批处理子系统为例，论述了批处理系统的设计与实现。该子系统承担成本文件生成、外部预约导入、待办状态更新、邮件与通知批量生成、集团主数据同步等作业，需在每日营业开始前完成全部处理，同时不得影响在线业务的响应时间。我在项目中担任系统架构师，负责批处理作业的划分与调度设计、容错与重跑机制设计以及与在线业务的资源隔离方案。针对批量作业与在线业务争抢数据库连接、跨时区环境下日期边界难以把握、异地数据同步缺少可观测性三大挑战，我从资源隔离、参数化基准日、指标主动推送三个维度进行设计：为批处理配置独立的读写数据源并按代码包路径隔离数据访问接口，使批量读取与在线交易在物理层面分离；把执行基准日与取得范围定义为显式作业参数而不依赖系统时钟，使补数重跑的结果完全可复现；由作业主动向监控网关推送执行指标，弥补了短生命周期进程难以被常规监控采集的缺陷。改造后，批量作业执行期间在线接口响应时间稳定在 320 毫秒以内，批量作业自身耗时下降约 30%。

正文

一、项目背景与批处理需求

BASE HOME 平台于 2024 年 3 月启动建设，2025 年 1 月上线运行，服务 H.I.S. 集团 300 余家营业门店与约 5000 名业务人员，日均处理预约约 3 万笔、消息与通知约 12 万条。我在项目中担任系统架构师，负责批处理作业的划分与调度设计、容错与重跑机制设计以及与在线业务的资源隔离方案。

平台的批处理需求来自业务的几个刚性环节。

第一是**成本结算**。营业门店的预约在出行前后需要产出成本文件，交由集团财务系统处理。这类作业数据量大、计算密集，且必须在财务结账日前完成。

第二是**外部数据导入**。平台需要定期从外部预约系统拉取预约数据并导入本地，同时从集团主数据平台同步国家、城市、航空公司、货币等主数据。

第三是**状态推进**。预约的待办事项、提醒状态等需要按日推进，例如出行前若干天触发提醒、逾期的待办自动关闭。

第四是**批量通知**。出行前的提醒通知、行程变更的批量告知等，需要在指定时刻批量生成并投递。

综合来看，平台的批处理子系统包含 10 个作业，其中既有每日执行的常规作业，也有按周、按月执行的周期性作业。这些作业的共同约束有两条：一是必须在**次日营业开始前完成**，因为营业开始后数据必须是最新的；二是**不得影响在线业务的响应时间**，因为批量执行时段与部分门店的营业时间存在重叠。

二、批处理架构的设计

作业划分的原则。我在划分作业时遵循三个原则。第一，按业务闭环划分，一个作业完成一件完整的业务事项，而不是按技术步骤拆分——例如“成本文件生成”作业内部包含了数据抽取、计算、文件生成与上传对象存储的完整过程，而不是拆成三个作业由调度器串联。这样做的理由是：拆得过细会使作业间的状态传递成为问题，一旦中间某个作业失败，难以判断重跑应从哪一步开始。第二，按数据边界划分，作业之间不共享数据表，避免一个作业的失败影响另一个作业的输入。第三，按执行频率与时机划分，把日次、周次、月次作业分开调度，使调度配置清晰。

调度策略的设计。平台的批处理不在应用内部定时触发——代码中没有任何内置的定时任务——而是由外部的调度链路驱动。完整的链路是：事件调度器按 cron 表达式触发，启动一个状态机编排流程，由状态机提交批处理计算任务，计算平台拉起容器执行作业，作业完成后进程退出。这条链路的一个关键设计是把“定时”与“计算”分离：调度器只负责在正确的时间点发起，计算平台只负责按需拉起容器，两侧都不需要常驻进程。同时，我把全部 cron 表达式都显式指定为东京时区，避免因平台默认时区与业务时区不一致而在错误的时刻触发——对营业系统而言，在营业时段内跑批处理是不可接受的。

作业的选择通过环境变量指定，作业参数通过容器的启动命令传入。这意味着同一个镜像可以承载全部作业，新增作业不需要新建部署单元，只需增加一条调度配置。

这里还有一个重要的架构决策：**批处理模块不启用 Web 容器**。理由是批处理不需要对外提供 HTTP 服务，若启用 Web 容器，不仅会占用端口与内存，还会使批处理模块具备被外部访问的能力，扩大了安全面。使批处理保持为纯粹的计算进程，既省资源，也降低了风险。

在调度粒度上，我把作业之间的依赖关系交给调度服务管理，而不是在作业代码内部互相调用。这样每个作业都可以独立触发、独立重跑，调度拓扑的调整也不需要修改代码。

容错与重跑机制。这是批处理设计中最关键的部分，我从三个层面进行设计。

第一层是**参数化**。我把执行基准日、取得范围、导入类型等关键参数定义为作业参数显式传入，而不依赖系统时钟取当前日期。这一设计的价值在跨时区部署时尤为突出：平台服务日本本部，而容器运行环境为协调世界时，若依赖系统时钟判断“今天”，在协调世界时的凌晨时段会取到与日本日期不一致的边界，导致漏处理或重复处理。参数化之后，补数时可以指定历史日期重跑，结果与当日运行完全一致；测试时可以固定参数，使结果可预期。

第二层是**幂等性**。批处理的失败重跑是常态，因此每个作业都必须能够安全地重复执行。我的做法是：写入前先按业务键判断是否已处理，已处理则跳过；对于汇总类作业，采用"先清理本次范围、再重新写入"的方式，而不是增量累加，避免重跑导致数据翻倍。对于对外投递类作业（如批量通知），则以投递状态跟踪表记录每一封邮件的投递结果（未投递 / 已投递），重跑时只处理未投递的记录，从根本上杜绝重复发送。

第三层是**可观测性**。批处理属于短生命周期的进程，作业结束后进程即退出，常规的进程存活监控无法采集其运行状态——这是批处理与在线服务在可观测性上最大的差异。为此我引入了指标推送机制：由作业在运行过程中主动把指标推送到 Prometheus 的 PushGateway，并把推送间隔设为 5 秒、关闭动作设为"进程退出前投递"，确保最后一批指标不会随进程退出而丢失。基于这些指标，运维人员可以设置告警规则——例如"作业在预定时刻未推送开始指标"即意味着作业未启动或被阻塞，这是比事后查看日志更及时的发现手段。

三、与在线业务的资源隔离

这一部分是本项目批处理设计中投入最大、收益也最显著的部分。

平台的成本文件生成、外部预约导入、待办状态更新等作业需要从生产库大量读取并回写结果，而在线预约接口同时也在访问同一数据库。在系统上线前的压测中，我们发现夜间批量作业执行期间，在线预约查询接口的响应时间由 300 毫秒劣化至 2.5 秒，已完全超出可用性目标。

经分析，根源是两者共享数据库连接池。批量扫描属于长时间、大数据量的连续查询，会长时间占用连接，而在线接口的请求需要立即获得连接才能响应。在连接池容量有限的情况下，批量的持续占用直接导致在线请求排队。

我采取了四项措施。

第一，**独立数据源**。为批处理配置独立的读写数据源：读取使用只读账号，写入使用写账号，并通过代码包路径隔离两套数据访问接口。这样，批量读取与在线交易在数据库连接层面彻底分离，批量作业无法挤占在线业务的数据源。

第二，**控制事务粒度**。我把事务隔离级别设置为读已提交，减少长事务持有锁的时间；同时把事务边界收敛到独立的服务类中，避免大事务回滚造成连带重放。

第三，**限制批量游标**。我把数据访问层的默认游标大小限制在 200 条，防止海量结果集一次性载入导致内存溢出——这是批处理最常见的故障模式，尤其在数据量随时间增长后才会暴露。需要注意的是，在线侧的同名参数取值为 1000，两侧刻意采用不同的取值：在线查询追求单次取回的效率，批量处理追求内存的可控。

第四，**收敛扫描范围**。对云数据库侧的扫描操作，我把扫描上限显式设置为一个有限值，为可能出现的误用设置硬边界，防止一次疏忽触发全表扫描。

改造后再次压测，批量作业执行期间在线接口的响应时间稳定在 320 毫秒以内；出人意料的是，批量作业自身的耗时反而下降了约 30%——原因是独立的只读数据源使读取不再与在线事务争抢锁资源，而且游标限制让内存回收更及时。

四、实施中遇到的问题及解决措施

问题一：批量作业峰值冲击在线服务。如上节所述，这是本项目批处理设计中最重要的问题，通过独立数据源、控制事务粒度、限制游标与收敛扫描范围四项措施解决。

问题二：主数据同步失败无从察觉。平台需要定期从集团主数据平台同步数据。上线初期曾出现主数据更新后业务侧仍在用旧值的情况，而运维人员无从知晓同步是否成功——因为同步作业以批处理方式运行，作业失败后没有任何通知。经排查，根源是缺少可观测手段，而非同步逻辑本身有缺陷。我为此补充了两项措施：一是把同

步作业的执行结果纳入监控，作业失败即触发告警；二是让同步作业的基准日期参数化，使补数可以精确指定日期范围重跑。改造后，主数据不一致的问题单数量下降约 90%。

问题三：批处理的重跑导致数据翻倍。 初期有个别汇总类作业采用增量累加的方式写入，一旦因网络抖动重跑，数据就会翻倍且难以察觉。我推动了写入方式的改造，把汇总类作业统一改为"先清理本次范围、再重新写入"，使重跑的结果与首次执行完全一致。这个问题的教训是：批处理的幂等性不能依赖"不会失败"这一假设，而必须在设计时就予以保证。

问题四：跨时区的日期边界。 平台服务日本本部，源数据库运行于日本标准时间，而容器运行环境为协调世界时，开发终端又是另一个时区。三套时区并存时，预约截止时间、邮件发送时刻等跨时区比较极易出错，后果是提前发送或漏发——对旅游业务而言这是不可接受的。我的处理原则是时间口径单一化：存储层统一使用协调世界时；展示层按业务方时区转换；转换动作集中到统一的格式化工具类，禁止在业务代码中散落直接取当前时间的调用。批处理侧则更为彻底——执行基准日与取得范围作为作业参数显式传入，不依赖系统时钟。

五、总结与展望

平台上线至今已稳定运行约 8 个月。批处理方面的成效主要体现在：全部作业均在次日营业开始前完成，未发生因批处理原因导致的营业延迟；批量作业执行期间在线接口响应时间稳定在 320 毫秒以内，批量作业自身耗时下降约 30%；主数据不一致的问题单数量下降约 90%。

目前仍存在三点不足。第一，作业之间的依赖关系由调度服务管理，但依赖关系的可视化程度不足，新成员理解调度拓扑需要阅读大量配置。第二，批处理的重跑目前需人工触发，尚未实现失败作业的自动恢复——这是有意为之的保守选择，因为误判自动重跑可能造成更大的影响，但代价是夜间故障的响应依赖人工值守。第三，批处理与在线业务虽然实现了数据源层面的隔离，但两者仍在同一数据库实例上，尚未实现实例级隔离。

后续计划完善调度拓扑的可视化，引入带条件判断的半自动重跑机制，并在数据量进一步增长时评估实例级隔离方案。

通过本项目，我对批处理系统的设计形成了三点认识。第一，批处理与在线服务的差异不仅在负载特征上，更在**可观测性模型**上——在线服务的健康状态可以通过持续监控获得，而批处理是"一次性事件"，必须由作业主动上报，否则失败的作业就是沉默的失败。第二，**幂等性不是优化项，而是批处理的必备属性**，因为失败重跑在批处理中是常态而非例外。第三，批量作业与在线业务的关系需要主动设计——它们共享同一个数据源是默认状态，而资源隔离是需要额外付出努力才能获得的能力；在项目初期就完成隔离，成本远低于上线后被动改造。

写作提示（不计入论文字数，背诵时请忽略本节）

待替换数字清单

位置	示例值	请替换为
门店数 / 业务人员数	300 余家 / 约 5000 名	真实数量
日均预约量 / 消息量	约 3 万笔 / 约 12 万条	真实数量
批处理作业数量	10 个	✓ 已核实
压测劣化情况	300ms → 2.5s	真实压测数据

位置	示例值	请替换为
改造后响应时间	320 毫秒以内	建议改用「90 分位 5 秒以内」（有测试报告支撑）
批量作业耗时下降	约 30%	建议改用「4 小时 47 分完成，达标 5 小时以内，吞吐 10 tps」（有实测支撑）
批量游标 / 扫描上限	200 条 / 10000	✔ 已核实
主数据问题单下降	约 90%	真实数据
稳定运行时长	约 8 个月	真实时长

本文字数

摘要约 350 字，正文约 2900 字，合计约 3250 字。

可复用要点

- 「批处理与在线服务的差异不仅在负载，更在可观测性模型」 是本篇最有价值的判断，绝大多数考生写不出来
- 「批量作业与在线业务共享数据源是默认状态，隔离才需要努力」 这句话点破了资源隔离的本质
- 参数化基准日的做法同时服务于"可重跑"与"跨时区正确"两个目标，是一石二鸟的设计点
- 若题目侧重"调度"，把第二节的调度策略展开，补充作业依赖拓扑、触发方式与并发控制
- 若题目侧重"大数据量处理"，把第三节的游标限制、分区读取、扫描边界展开